Artificial Intelligence & Machine Learning Lab

Complete Practical Source Codes & Program Outputs Reference Manual

1. Linear Regression Analysis (Salary vs Experience)

Simple linear regression modeling, train-test splitting, metrics calculation (MSE, R^2), and edge case testing.

SOURCE CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. File Upload & Dataset Loading
files.upload()
df = pd.read_csv("Salary_Data.csv")

print("==== SALARY DATASET ====")
print(df.head(), "\nRows: ", len(df))

# Features and Target
X = df[["YearsExperience"]]
y = df["Salary"]

# Base Model Fitting
model = LinearRegression().fit(X, y)

# ----------------- TEST CASE 1 -----------------
print("\n--- TEST CASE 1 ---")
print("Slope: ", model.coef_[0], " Intercept: ", model.intercept_)
print("RESULT: ", "PASS" if model.coef_[0] > 0 else "FAIL")

# ----------------- TEST CASE 2 -----------------
# Train/Test Split & Evaluation
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)

m = LinearRegression().fit(X_tr, y_tr)
pred = m.predict(X_te)

mse = mean_squared_error(y_te, pred)
r2 = r2_score(y_te, pred)

print("\n--- TEST CASE 2 ---")
print("Actual: ", y_te.values)
print("Predicted: ", np.round(pred, 2))
print("MSE: ", round(mse, 2), " R2: ", round(r2, 3))
print("RESULT: ", "PASS" if r2 > 0.5 else "FAIL")

# ----------------- TEST CASE 3 -----------------
pred_7 = model.predict([[7]])
print("\n--- TEST CASE 3 ---")
print("7 Years Salary: ", round(pred_7[0], 2))

# ----------------- TEST CASE 4 -----------------
# Constant Target Check
yc = np.full_like(y, fill_value=y.mean())
mc = LinearRegression().fit(X, yc)
```

```
print("\n--- TEST CASE 4 ---")
print("Constant Target Slope: ", mc.coef_[0])
print("RESULT: ", "PASS" if abs(mc.coef_[0]) < 1e-6 else "FAIL")

# ----------------- TEST CASE 5 -----------------
# Outlier Robustness Check
y_outlier = y.copy()
y_outlier.iloc[0] = y.max() * 5
mo = LinearRegression().fit(X, y_outlier)

normal_mse = mean_squared_error(y, model.predict(X))
outlier_mse = mean_squared_error(y_outlier, mo.predict(X))

print("\n--- TEST CASE 5 ---")
print("Normal MSE: ", normal_mse)
print("Outlier MSE: ", outlier_mse)

# ----------------- VISUALIZATION -----------------
plt.scatter(X, y, label="Data")
plt.plot(X, model.predict(X), color="red", label="Regression Line")
plt.title("Salary vs Experience")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.legend()
plt.show()
```

PROGRAM OUTPUT

```
Salary_Dataset.csv(text/csv) - 454 bytes, last modified: 9/15/21
Saving Salary_Data.csv to Salary_Data.csv

===== SALARY DATASET =====
   Experience_Years    Salary
0               1.1     39343
1               1.2     42774
2               1.3     46205
3               1.5     37731
4               2.0     43525
Rows: 30

--- TEST CASE 1 ---
Slope: 9523.65000 Intercept: 25673.075760
RESULT: PASS

--- TEST CASE 2 ---
Actual: [61111 56957 55794 93940 43525 57189 112635]
Predicted: [69052.39 64348.37 64348.38 83161.64 45321.71 61525.97 115033.35]
MSE: 49077731.17  R2: 0.907
RESULT: PASS

--- TEST CASE 3 ---
7 Years Salary: 92338.57

--- TEST CASE 4 ---
Constant Target Slope: 0.0
RESULT: PASS

--- TEST CASE 5 ---
Normal MSE: 28960383.65277627
Outlier MSE: 457851223.82289314
```

2. Decision Tree Classification (ID3 with Entropy)

PlayTennis dataset classification using DecisionTreeClassifier with entropy criterion, Label Encoding, and sample test evaluation.

SOURCE CODE

```
import pandas as pd
from sklearn import preprocessing
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

# ----------------- DATASET DEFINITION -----------------
data = {
    'Outlook': [
        'Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain',
        'Rain', 'Overcast', 'Sunny', 'Sunny', 'Rain',
        'Sunny', 'Overcast', 'Overcast', 'Rain'
    ],
    'Temperature': [
        'Hot', 'Hot', 'Hot', 'Mild', 'Cool',
        'Cool', 'Cool', 'Mild', 'Cool', 'Mild',
        'Mild', 'Mild', 'Hot', 'Mild'
    ],
    'Humidity': [
        'High', 'High', 'High', 'High', 'Normal',
        'Normal', 'Normal', 'High', 'Normal', 'Normal',
        'Normal', 'High', 'Normal', 'High'
    ],
    'Wind': [
        'Weak', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Weak', 'Weak',
        'Strong', 'Strong', 'Weak', 'Strong'
    ],
    'Play': [
        'No', 'No', 'Yes', 'Yes', 'Yes',
        'No', 'Yes', 'No', 'Yes', 'Yes',
        'Yes', 'Yes', 'Yes', 'No'
    ]
}

# Create DataFrame
df = pd.DataFrame(data)
print("Original Dataset:\n")
print(df)

# ----------------- LABEL ENCODING -----------------
encoders = {}
for column in df.columns:
    encoders[column] = LabelEncoder()
    df[column] = encoders[column].fit_transform(df[column])

# Features and Target
X = df[['Outlook', 'Temperature', 'Humidity', 'Wind']]
y = df['Play']

# ----------------- MODEL TRAINING -----------------
model = DecisionTreeClassifier(criterion='entropy')
model.fit(X, y)

# ----------------- TEST CASES -----------------
test_cases = pd.DataFrame({
    'Outlook': ['Sunny', 'Overcast', 'Rain', 'Sunny', 'Rain'],
    'Temperature': ['Hot', 'Cool', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'Normal', 'High', 'Normal', 'Normal'],
    'Wind': ['Strong', 'Weak', 'Strong', 'Weak', 'Weak']
})
```

```
test_encoded = test_cases.copy()
for col in test_cases.columns:
    test_encoded[col] = encoders[col].transform(test_cases[col])

predictions = model.predict(test_encoded)

print("\nTest Cases and Predictions")
print("-------------------------")
for i in range(len(test_cases)):
    outlook = test_cases.iloc[i]['Outlook']
    temperature = test_cases.iloc[i]['Temperature']
    humidity = test_cases.iloc[i]['Humidity']
    wind = test_cases.iloc[i]['Wind']
    result = encoders['Play'].inverse_transform([predictions[i]])[0]

    print(f"Test case {i + 1}:")
    print(f"  Outlook = {outlook}, Temperature = {temperature}, Humidity = {humidity}, Wind = {wind}
--> Play = {result}")

# ----------------- ACCURACY & EVALUATION -----------------
train_prediction = model.predict(X)
accuracy = (train_prediction == y).mean()
print(f"\nTraining Accuracy: {accuracy * 100:.2f}%")

# ----------------- VISUALIZATION -----------------
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=['No', 'Yes'],
    filled=True
)
plt.title("Decision Tree Classification using Entropy")
plt.show()
```

PROGRAM OUTPUT

```
Original Dataset:

     Outlook Temperature Humidity    Wind Play
0      Sunny         Hot     High    Weak   No
1      Sunny         Hot     High  Strong   No
2   Overcast         Hot     High    Weak  Yes
3       Rain        Mild     High    Weak  Yes
4       Rain        Cool   Normal    Weak  Yes
5       Rain        Cool   Normal  Strong   No
6   Overcast        Cool   Normal  Strong  Yes
7      Sunny        Mild     High    Weak   No
8      Sunny        Cool   Normal    Weak  Yes
9       Rain        Mild   Normal    Weak  Yes
10     Sunny        Mild   Normal  Strong  Yes
11  Overcast        Mild     High  Strong  Yes
12  Overcast         Hot   Normal    Weak  Yes
13      Rain        Mild     High  Strong   No

Test Cases and Predictions
-------------------------
Test case 1:
  Outlook = Sunny, Temperature = Hot, Humidity = High, Wind = Strong --> Play = No
Test case 2:
  Outlook = Overcast, Temperature = Cool, Humidity = Normal, Wind = Weak --> Play = Yes
Test case 3:
  Outlook = Rain, Temperature = Mild, Humidity = High, Wind = Strong --> Play = No
Test case 4:
  Outlook = Sunny, Temperature = Hot, Humidity = Normal, Wind = Weak --> Play = Yes
Test case 5:
```

```
  Outlook = Rain, Temperature = Mild, Humidity = Normal, Wind = Weak --> Play = Yes

Training Accuracy: 100.00%
```

3. K-Means Clustering on Iris Dataset

Unsupervised clustering on Iris flower measurements, testing varying cluster numbers (K=2, K=3), iteration convergence, and data duplication effects.

SOURCE CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.cluster import KMeans

# ----------------- LOAD DATASET -----------------
iris = datasets.load_iris()
x = pd.DataFrame(iris.data)
x.columns = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

plt.figure(figsize=(15, 10))

# ----------------- TEST CASE 1: TWO CLUSTERS -----------------
model1 = KMeans(n_clusters=2, random_state=0)
model1.fit(x)

plt.subplot(2, 3, 1)
plt.scatter(x['petal_length'], x['petal_width'], c=model1.labels_, cmap='viridis')
plt.scatter(model1.cluster_centers_[:, 2], model1.cluster_centers_[:, 3], color='red', marker='x',
s=100)
plt.title("TC1: 2 Centroids")

# ----------------- TEST CASE 2: THREE CLUSTERS (ORIGINAL) -----------------
model2 = KMeans(n_clusters=3, random_state=0)
model2.fit(x)

plt.subplot(2, 3, 2)
plt.scatter(x['petal_length'], x['petal_width'], c=model2.labels_, cmap='viridis')
plt.scatter(model2.cluster_centers_[:, 2], model2.cluster_centers_[:, 3], color='red', marker='x',
s=100)
plt.title("TC2: 3 Centroids")

# ----------------- TEST CASE 3: CENTROID CONVERGENCE -----------------
model3 = KMeans(n_clusters=3, max_iter=300, random_state=0)
model3.fit(x)

plt.subplot(2, 3, 3)
plt.scatter(x['petal_length'], x['petal_width'], c=model3.labels_, cmap='viridis')
plt.scatter(model3.cluster_centers_[:, 2], model3.cluster_centers_[:, 3], color='red', marker='x',
s=100)
plt.title("TC3: Converged")

print("\nTC3: Convergence Test")
print("Number of iterations: ", model3.n_iter_)
print("Centroids: ")
print(model3.cluster_centers_)

# ----------------- TEST CASE 4: K > DATA POINTS -----------------
plt.subplot(2, 3, 4)
try:
    x_small = x.head(5)
    model4 = KMeans(n_clusters=10, random_state=0)
    model4.fit(x_small)
```

```
except ValueError:
    plt.text(0.2, 0.5, "Error: k > data points", fontsize=10)
    plt.title("TC4: Error")

# ----------------- TEST CASE 5: DUPLICATE POINTS -----------------
x_dup = pd.concat([x, x.iloc[:50], x.iloc[:50], x.iloc[:50]])
print("\nOriginal Records: ", len(x))
print("Records After Adding Duplicates: ", len(x_dup))

model5 = KMeans(n_clusters=3, random_state=0)
model5.fit(x_dup)

plt.subplot(2, 3, 5)
plt.scatter(x_dup['petal_length'], x_dup['petal_width'], c=model5.labels_, cmap='viridis')
plt.scatter(model5.cluster_centers_[:, 2], model5.cluster_centers_[:, 3], color='red', marker='x',
s=100)
plt.title("TC5: Many Duplicate Points")

plt.tight_layout()
plt.show()
```

PROGRAM OUTPUT

```
TC3: Convergence Test
Number of iterations:  3
Centroids:
[[5.9016129  2.7483871  4.39354839 1.43387097]
 [5.006      3.428      1.462      0.246     ]
 [6.85       3.07368421 5.74210526 2.07105263]]

Original Records:  150
Records After Adding Duplicates:  300
```

4. Support Vector Machine (SVM) Classification

Linear SVM binary classifier on linearly separable iris classes, margin boundary verification, decision function evaluation, and 2D hyperplane plot.

SOURCE CODE

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# ----------------- LOAD DATASET -----------------
iris = load_iris()
x, y = iris.data, iris.target

# ============================================================
# TC1: Linearly Separable Data (Classes 0 and 1 only)
# ============================================================
x1, y1 = x[y != 2], y[y != 2]

model1 = SVC(kernel='linear')
model1.fit(x1, y1)
print("TC1 Accuracy :", accuracy_score(y1, model1.predict(x1)))

# ============================================================
# TC2: Two-Class Dataset (Train-Test Split)
# ============================================================
x_train, x_test, y_train, y_test = train_test_split(
```

```
    x1, y1, test_size=0.2, random_state=42
)

model2 = SVC(kernel='linear')
model2.fit(x_train, y_train)
print("TC2 Accuracy :", accuracy_score(y_test, model2.predict(x_test)))

# ============================================================
# TC3: Test Point Near Margin
# ============================================================
model3 = SVC(kernel='linear')
model3.fit(x1, y1)

# Point from the dataset located near the decision boundary
point = [x1[23]]

print("TC3 predicted class: ", model3.predict(point)[0])
print("TC3 decision value : ", model3.decision_function(point)[0])

# ============================================================
# TC4: Overlapping Classes (Added Noise)
# ============================================================
np.random.seed(42)
x4 = x1 + np.random.normal(0, 0.8, x1.shape)

model4 = SVC(kernel='linear')
model4.fit(x4, y1)
print("TC4 Accuracy :", accuracy_score(y1, model4.predict(x4)))

# ============================================================
# TC5: Single-Class Data
# ============================================================
x5, y5 = x[y == 0], y[y == 0]

try:
    model5 = SVC(kernel='linear')
    model5.fit(x5, y5)
except ValueError:
    print("TC5: SVM needs at least two classes.")

# ============================================================
# VISUALIZATION (Decision Boundary & Margins)
# ============================================================
xg = x1[:, :2]
graph_model = SVC(kernel='linear')
graph_model.fit(xg, y1)

plt.scatter(xg[y1 == 0, 0], xg[y1 == 0, 1], label="Class 0")
plt.scatter(xg[y1 == 1, 0], xg[y1 == 1, 1], label="Class 1")

xx = np.linspace(xg[:, 0].min(), xg[:, 0].max(), 100)
w = graph_model.coef_[0]
b = graph_model.intercept_[0]

yy = -(w[0] * xx + b) / w[1]
plt.plot(xx, yy, linewidth=2, label="Hyperplane")

yy_lower = -(w[0] * xx + b + 1) / w[1]
plt.plot(xx, yy_lower, '--', label="margin - 1")

plt.scatter(
    graph_model.support_vectors_[:, 0],
    graph_model.support_vectors_[:, 1],
    facecolors='none',
    edgecolors='black',
    s=100
```

```
)

plt.xlabel("Sepal length")
plt.ylabel("Sepal width")
plt.title("SVM Classification")
plt.legend()
plt.grid(True)
plt.show()
```

PROGRAM OUTPUT

```
TC1 Accuracy : 1.0
TC2 Accuracy : 1.0
TC3 predicted class:  0
TC3 decision value :  -0.9995884524518365
TC4 Accuracy : 0.98
TC5: SVM needs at least two classes.
```

5. Bayesian Network Inference

Probabilistic inference on the classical Burglary-Earthquake-Alarm network using Conditional Probability Tables (CPT).

SOURCE CODE

```
# Simple Bayesian Network
# Burglary and Earthquake probabilities
P_B = 0.002
P_E = 0.001

# Alarm Conditional Probability Table (CPT)
alarm = {
    (True, True): 0.95,
    (True, False): 0.94,
    (False, True): 0.29,
    (False, False): 0.001
}

# David and Sophia call probabilities
P_David = 0.90
P_Sophia = 0.70

# ----------------- TEST CASE 1 -----------------
print("Test case 1")
B = False
E = False
A = True
D = True
S = True

p = (1 - P_B) * (1 - P_E) * alarm[(B, E)] * P_David * P_Sophia
print("Probability = ", p)

# ----------------- TEST CASE 2 -----------------
print("\nIn Test case 2")
B = True
E = False
A = True

p = P_B * (1 - P_E) * alarm[(B, E)] * P_Sophia
print("Probability = ", p)

# ----------------- TEST CASE 3 -----------------
print("\nIn Test case 3")
B = False
```

```
E = True
A = True

p = (1 - P_B) * P_E * alarm[(B, E)] * P_David * (1 - P_Sophia)
print("Probability = ", p)

# ----------------- TEST CASE 4: MISSING CPT -----------------
print("\nIn Test case 4")
if (False, False) not in alarm:
    print("missing CPT value")
else:
    print("All CPT values are present")

# ----------------- TEST CASE 5: INVALID PROBABILITY -----------------
print("\nIn Test case 5")
x = 1.5
if x < 0 or x > 1:
    print("Invalid probability: ", x)
else:
    print("Valid probability")
```

PROGRAM OUTPUT

```
Test case 1
Probability =  0.0006281112599999999

In Test case 2
Probability =  0.001314684

In Test case 3
Probability =  7.81434e-05

In Test case 4
All CPT values are present

In Test case 5
Invalid probability:  1.5
```

6. Knowledge Base & Propositional Logic Inference

Automated reasoning using SymPy logic solver (satisfiable), testing logical implication, exclusive OR (XOR), and contradiction detection.

SOURCE CODE

```
from sympy import symbols, Not, Implies, Xor
from sympy.logic.inference import satisfiable

# ----------------- PROPOSITIONAL SYMBOLS -----------------
rain = symbols("rain")
harry_visited_hagrid = symbols("harry_visited_hagrid")
harry_visited_dumbledore = symbols("harry_visited_dumbledore")

# ----------------- INFERENCE FUNCTION -----------------
def run_test(test_name, kb):
    print("\n" + "=" * 50)
    print(test_name)
    print("=" * 50)

    result = satisfiable(kb)
    if result is False:
        print("Contradiction (Not satisfiable)")
    else:
        print("Satisfiable model: ", result)
```

```
        if rain in result:
            if result[rain]:
                print("It rained today")
            else:
                print("There was no rain today.")
        else:
            print("Rain cannot be determined.")

# ----------------- TEST CASE 1 -----------------
kb1 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_dumbledore
)
run_test("Test Case 1", kb1)

# ----------------- TEST CASE 2 -----------------
kb2 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_dumbledore
    & rain
)
run_test("Test Case 2", kb2)

# ----------------- TEST CASE 3 -----------------
kb3 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_hagrid
    & rain
)
run_test("Test Case 3", kb3)

# ----------------- TEST CASE 4 -----------------
kb4 = (
    Implies(Not(rain), harry_visited_hagrid)
    & Xor(harry_visited_hagrid, harry_visited_dumbledore)
    & harry_visited_hagrid
    & harry_visited_dumbledore
)
run_test("Test Case 4", kb4)

# ----------------- TEST CASE 5 -----------------
kb5 = harry_visited_hagrid
run_test("Test Case 5", kb5)
```

PROGRAM OUTPUT

```
==================================================
Test case 1
==================================================
Satisfiable model:  {harry_visited_dumbledore: True, rain: True, harry_visited_hagrid: False}
It rained today

==================================================
Test case 2
==================================================
Satisfiable model:  {harry_visited_dumbledore: True, rain: True, harry_visited_hagrid: False}
It rained today

==================================================
Test case 3
==================================================
Satisfiable model:  {harry_visited_hagrid: True, rain: True, harry_visited_dumbledore: False}
It rained today
```

```
================================================
Test case 4
================================================
Contradiction (Not satisfiable)

================================================
Test case 5
================================================
Satisfiable model:  {harry_visited_hagrid: True}
Rain cannot be determined.
```

7. A* Search Algorithm

Heuristic search pathfinding finding optimal path f(n) = g(n) + h(n), tested under admissible, misleading, and disconnected graphs.

SOURCE CODE

```
def a_star(graph, h, start, goal):
    open_list = [start]
    closed = set()
    g = {start: 0}
    parent = {start: start}

    while open_list:
        # Find node in open_list with lowest f(n) = g(n) + h(n)
        n = open_list[0]
        for node in open_list:
            if g[node] + h[node] < g[n] + h[n]:
                n = node

        # Goal check and path reconstruction
        if n == goal:
            path = []
            while parent[n] != n:
                path.append(n)
                n = parent[n]
            path.append(n)
            path.reverse()
            print("path :", path)
            print("cost :", g[goal])
            return

        # Explore neighbors
        for m, cost in graph[n]:
            new_cost = g[n] + cost
            if m not in open_list and m not in closed:
                open_list.append(m)
                parent[m] = n
                g[m] = new_cost
            elif new_cost < g.get(m, float('inf')):
                g[m] = new_cost
                parent[m] = n
                if m in closed:
                    closed.remove(m)
                    open_list.append(m)

        open_list.remove(n)
        closed.add(n)

    print("path not found")


# ----------------- TEST CASE 1: ADMISSIBLE HEURISTIC -----------------
print("Test Case 1: admissible heuristic")
```

```
graph1 = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [('G', 2)],
    'E': [('G', 4)],
    'F': [('G', 1)],
    'G': []
}
h1 = {'A': 6, 'B': 4, 'C': 4, 'D': 2, 'E': 3, 'F': 1, 'G': 0}
a_star(graph1, h1, 'A', 'G')

# ----------------- TEST CASE 2: MULTIPLE POSSIBLE PATHS -----------------
print("\nIn Test Case 2: multiple possible paths")
graph2 = {
    'S': [('A', 2), ('B', 4)],
    'A': [('G', 5)],
    'B': [('G', 2)],
    'G': []
}
h2 = {'S': 3, 'A': 2, 'B': 1, 'G': 0}
a_star(graph2, h2, 'S', 'G')

# ----------------- TEST CASE 3: DIRECT START-GOAL CONNECTION -----------------
print("\nIn Test Case 3: Direct-start Goal connection")
graph3 = {
    'A': [('G', 2), ('B', 4)],
    'B': [('G', 3)],
    'G': []
}
h3 = {'A': 2, 'B': 3, 'G': 0}
a_star(graph3, h3, 'A', 'G')

# ----------------- TEST CASE 4: GOAL NOT REACHABLE -----------------
print("\nIn Test Case 4: Goal not reachable")
graph4 = {
    'A': [('B', 2)],
    'B': [('C', 3)],
    'C': [],
    'G': []
}
h4 = {'A': 4, 'B': 3, 'C': 2, 'G': 0}
a_star(graph4, h4, 'A', 'G')

# ----------------- TEST CASE 5: MISLEADING HEURISTIC -----------------
print("\nIn Test Case 5: misleading heuristic")
graph5 = {
    'S': [('A', 1), ('B', 2)],
    'A': [('G', 10)],
    'B': [('G', 2)],
    'G': []
}
h5 = {'S': 1, 'A': 0, 'B': 100, 'G': 0}
a_star(graph5, h5, 'S', 'G')
```

PROGRAM OUTPUT

```
Test Case 1: Admissible heuristic
path : ['A', 'B', 'E', 'G']
cost : 6

In Test Case 2: multiple possible path
path : ['S', 'B', 'G']
cost : 6

In Test Case 3: Direct start-goal connection
```

```
path : ['A', 'G']
cost : 2

Test Case 4: Goal not found
path not found

Test Case 5: misleading heuristic
path : ['S', 'A', 'G']
cost : 11
```